

Observability Architecture for Enterprise Agentic AI Agents

Rich Lee

S P E A K E R



李琦 Rich

Architect

Cathay Financial Holding DDT Center

- Cloud-Native Application
- Cloud Security
- Event-driven Architecture

Agenda

The Why

- Silent failures & probabilistic systems
 - Application vs Agent observability
-

The How

- Three signals & Golden Signals for agents
 - Evaluation
-

Architecting

- Three signals & Golden Signals for agents
- Evaluation

Agent systems fail silently

Traditional System Error

500 Server Error

Agent System Failure

200 OK

- Hallucination
- Soft Tool Failures

Web service failure

500 Internal Server Error
Loud, structured, indexable.

Agent failure

Silent. Probabilistic. Looks correct.
Surfaces in user feedback weeks later.

Cost of blindness

\$ token spend, eroded trust,
compliance exposure, runaway loops.
loops.

Traditional Application vs AI Agent

Observability for agents must capture **reasoning, tool use, retrieval, and intent**

DETERMINISTIC · Traditional app

- Fixed control flow, known endpoints
- Errors carry explicit codes
- Latency is the main SLO
- Schema-stable inputs & outputs

PROBABILISTIC · Agentic system

- LLM picks the next step at runtime
- Same input ≠ same trajectory
- Failures are semantic, not status-code
- Quality + cost + safety are all SLOs
- Free-text in, free-text + tool calls out

OWASP TOP 10 for Agentic AI 2026

► Shift in threat model

- From talking AI (chatbots) to acting AI (autonomous agents)
- Classic LLM Top 10 → mostly info leakage and direct prompt injection
- Agentic Top 10 → planning, tool calls, data access, memory, identity

► New top-of-mind risks

- ASI01 Goal Hijacking / Indirect Prompt Injection
- ASI02 Tool Misuse
- ASI03 Privilege Escalation across tools / sub-agents
- ASI04 Memory & Context Poisoning
- ASI09 Excessive Agency (no human-in-the-loop where it matters)



What is Agent Observability?

For **agents**, **traces** are most important, since they reveal reasoning and decision-making.

Metrics

- cost /session success rate
- P95 latency
- eval scores over time

Logs

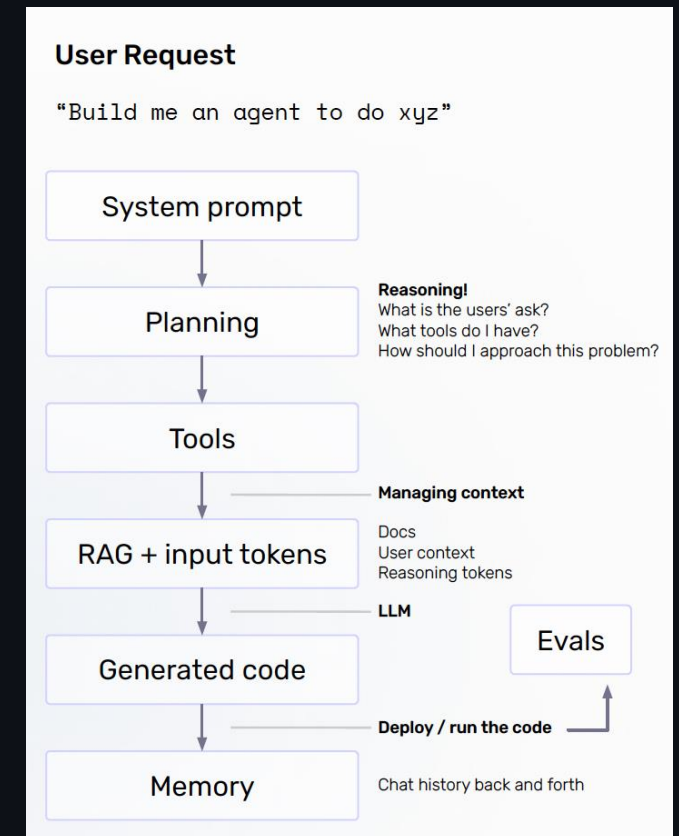
- errors
- guardrail triggers

Traces & Spans

The full causal tree of one user turn

- Prompts and Reasoning
- tool calls
- retrieved docs
- sub-agents.

AI Agent Observability



The trace is your documentation in AI agents

In traditional software, the code is your documentation.

```
class OrderProcessor:
    def __init__(self, inventory_db, payment_gateway):
        ...

    def process_order(self, order_id: str, user_id: str, items: list, payment_info: dict):
        # Step 1: Deterministic Input Validation
        if not items:
            return {"status": "REJECTED", "reason": "Empty cart"}

        # Step 2: Strict Conditional Branching (Inventory Check)
        for item in items:
            if not self.inventory_db.is_in_stock(item['id'], item['quantity']):
                # Path A: Explicitly defined failure path
                return {"status": "FAILED", "reason": f"Item {item['id']} out of stock"}

        # Step 3: Pure Mathematical Calculation
        subtotal = sum(item['price'] * item['quantity'] for item in items)
        tax = subtotal * 0.05 # Fixed 5% tax rate
        total_price = subtotal + tax

        # Step 4: External State Mutation
        payment_success = self.payment_gateway.charge(user_id, total_price, payment_info)
```

In AI agents, the trace is your documentation.

```
from google.adk import Agent
from google.adk.tools import google_search

agent = Agent(
    name="researcher",
    model="gemini-flash-latest",
    instruction="You are a helpful ai architect...",
    tools=[search_tool, analysis_tool, visualization_tool],
)
```

Trace ID 9f95c819352cfc60a32d291635dd490a

Trace Status	Total Cost	Latency
OK	\$0.05	24.5s

Trace

- code_based_agent
 - router_call
 - ChatCompletion 264
 - generate_and_run_sql_query
 - ChatCompletion 922
 - run_sql_query
 - router_call
 - ChatCompletion 1,110
 - data_analyzer
 - data_analysis_tool
 - ChatCompletion 1
 - router_call

Input

What trends do you see in my trace latency?

Output

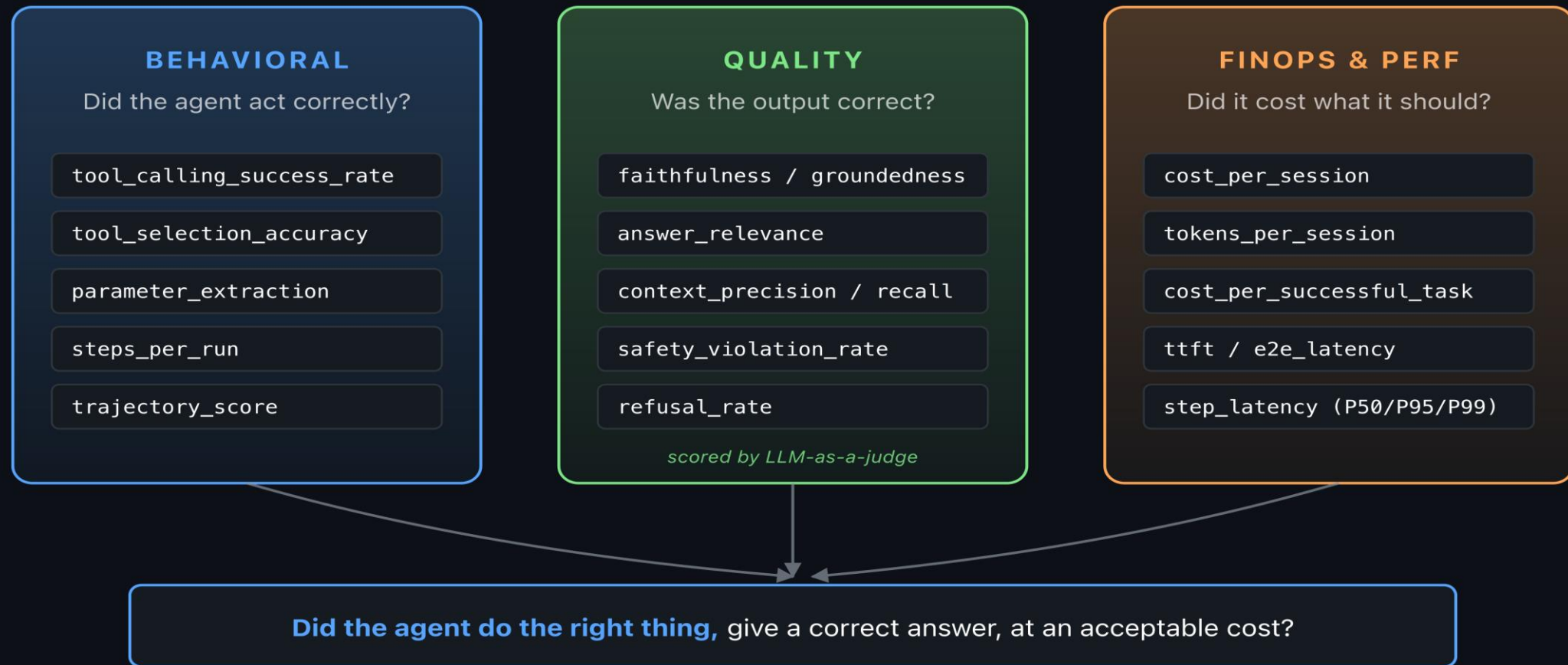
Trends in Trace Latency

Based on the analysis of your trace latency data, here are some key trends and insights:

- Retrieve Operation:**
 - Most operations have a latency of 0, indicating these operations are generally quick.
 - Occasionally, latencies of 1 are observed, but these are infrequent.

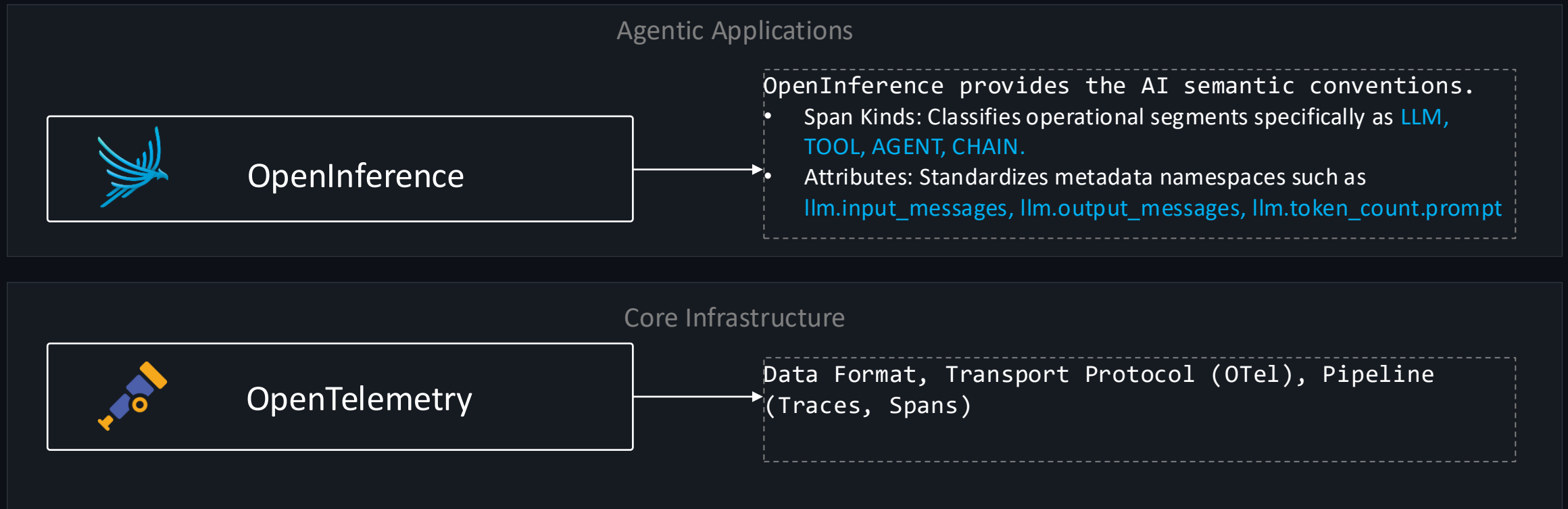
The Golden Signals for Agents

Traditional SRE has 4 golden signals. Agents need three families

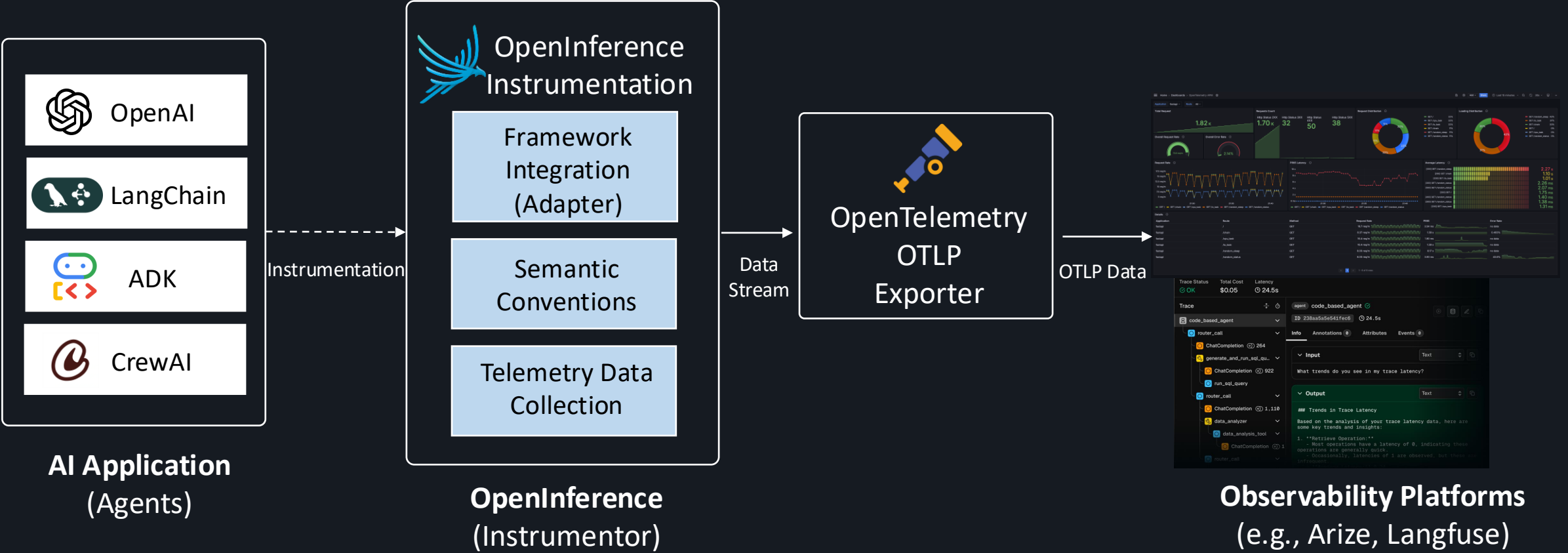


Standards – OTel and OI (OpenInference)

OpenInference built on top of OTel and it designed specifically for Agentic applications



OpenInference Reference Architecture



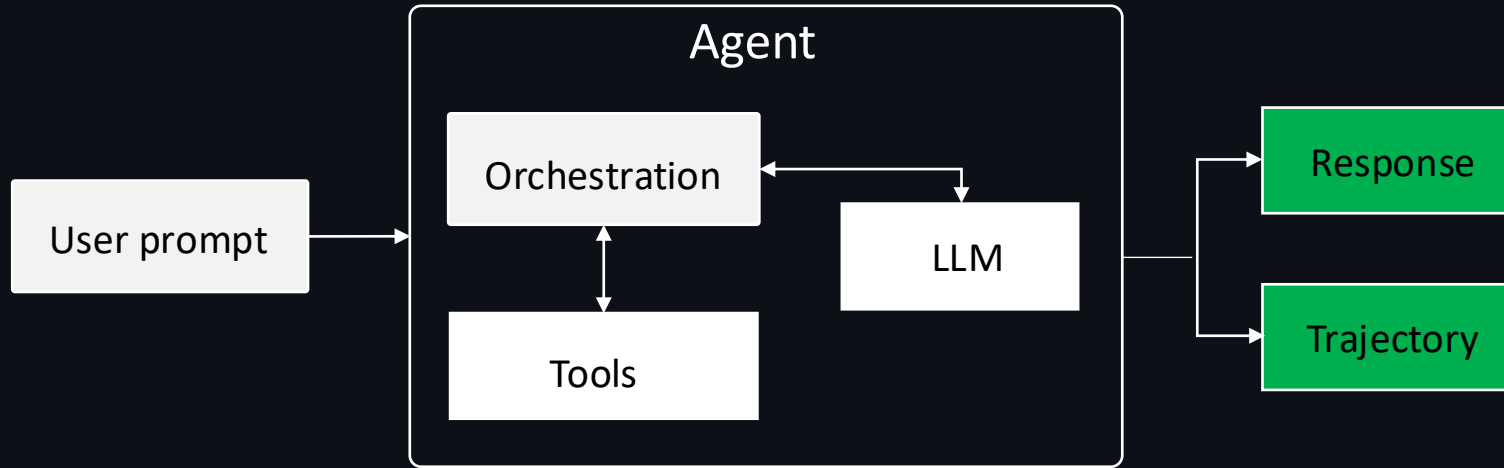
Why Agent Evaluation Is Different

Agent evaluation must assess autonomy, reasoning, tool use, and how the agent handles unpredictable situations.

	Traditional Software Testing	LLM Model Evaluation	LLM Agent Evaluation
Core Unit	Function, Module, Service	Foundation LLM	Entire System (LLM, Tools, Prompts, Memory)
Behavior	Deterministic & Predictable	Probabilistic (Single Response)	Probabilistic & Emergent (Multi-Step Trajectory)
Key Metrics	Pass/Fail, Code Coverage	Accuracy, Perplexity, BLEU/ROUGE	Task Completion, Tool Use Success, Reasoning Coherence, Cost

[agent-factory-recap](#)

What to Evaluate



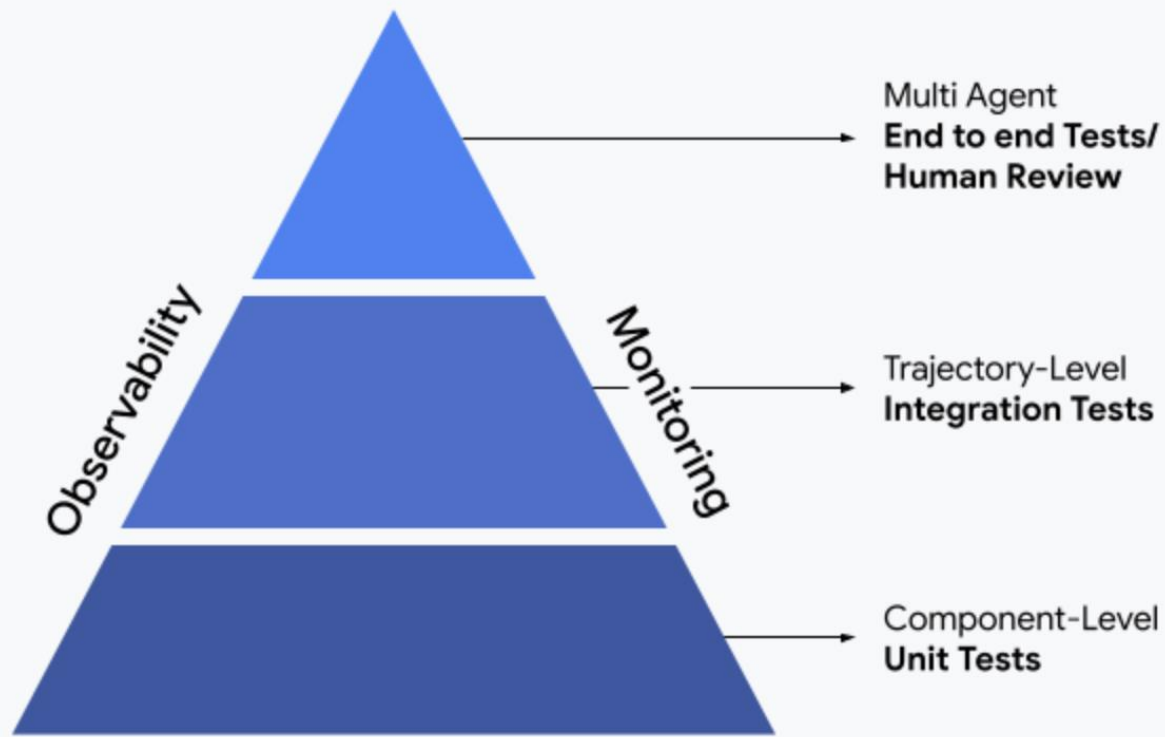
Outcome Evaluation

- **Scores the final result**
 - Answer, report, code, decision
- **Programmatic checks**
 - Regex, JSON schema, exact match
- **LLM-as-a-Judge**
 - Fluency, faithfulness, helpfulness
- **Rubric-based judging**
 - Structured checklists → explainable scoring

Trajectory Evaluation

- **Scores the path the agent took**
 - Did it pick the right tool?
 - Did it loop or backtrack?
- **Step-level metrics**
 - Tool-selection accuracy
 - Plan adherence vs golden trace
- **Catches silent failures**
 - Right answer via wrong reasoning is still a bug

3-Tier Testing Framework



Tier 3 - E2E Tests

HITL, Annotation

Tier 2 - Integration Tests

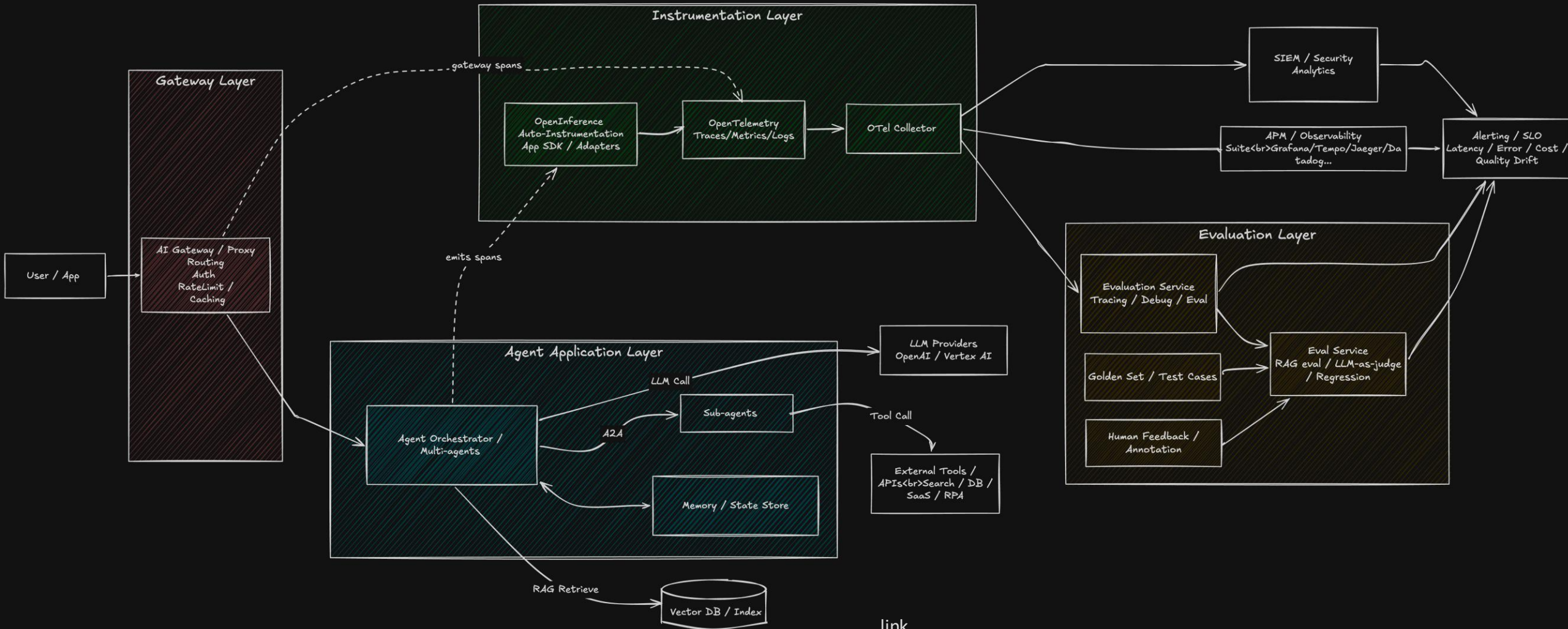
Evaluator, LLM-as-a-Judge

Tier 1 - Unit Tests

Framework build-in local eval

Observability Architecture for Enterprise Agentic AI

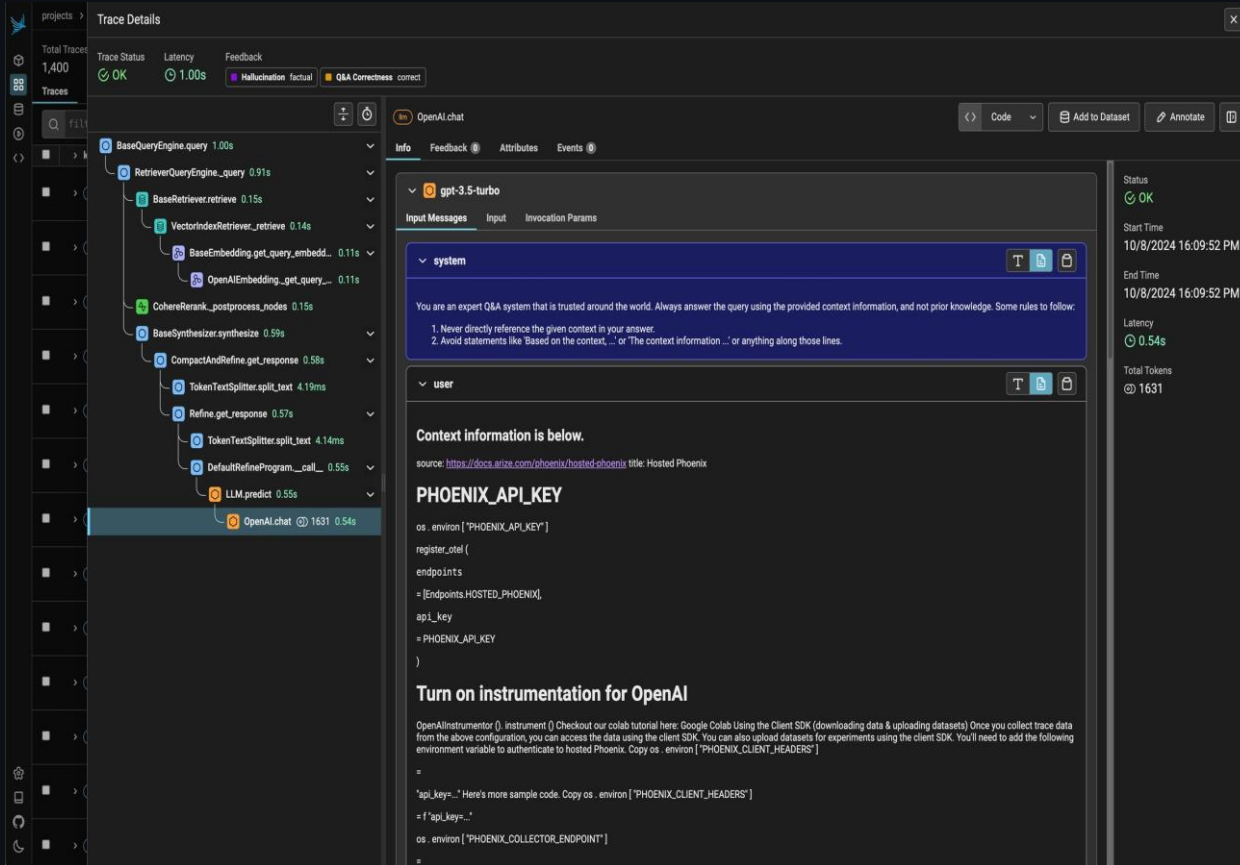
AI Agent Observability & Evaluation Architecture



[link](#)

Agent Observability Platform - Arize Phoenix

Open-source AI observability + evaluation platform from Arize AI.



source: arize.com/docs/phoenix/tracing/llm-traces

Tracing

OpenInference-native span viewer.

Evaluation

Built-in hallucination, relevance, trajectory evals.

Prompts

Version, A/B, replay against
past traces.

Experiments

Run agent changes vs datasets, diff scores.

`pip install arize-phoenix` · 30+ auto-instrumentors incl. **Google ADK**, LangChain, LlamaIndex, CrewAI, DSPy.

Google ADK & Arize Phoenix Integration

```
from phoenix.otel import register
from openinference.instrumentation.google_adk \
    import GoogleADKInstrumentor

# 1. point spans at your Phoenix instance
tracer_provider = register(
    project_name="devops-agent-project",
    endpoint="http://phoenix:6006/v1/traces",
)

# 2. auto-instrument every ADK agent + tool call
GoogleADKInstrumentor().instrument(
    tracer_provider=tracer_provider,
)

# --- your normal ADK code below, unchanged ---
from google.adk.agents import Agent
root_agent = Agent(
    name="devops_agent",
    model="gemini-3.5-flash",
    tools=[google_search, lookup_account],
)
```

The screenshot displays the Arize Phoenix web interface for a specific trace. The top bar shows the trace ID: `c346819294357560c2e665d7f8fe6fec` and the session ID: `4f287639-b1d5-44d1-9296-90d5e4150a66`. The trace status is 'OK' with a latency of 4.83s and a start time of 6/18/2025, 5:08:13 PM.

The trace tree on the left shows the following structure:

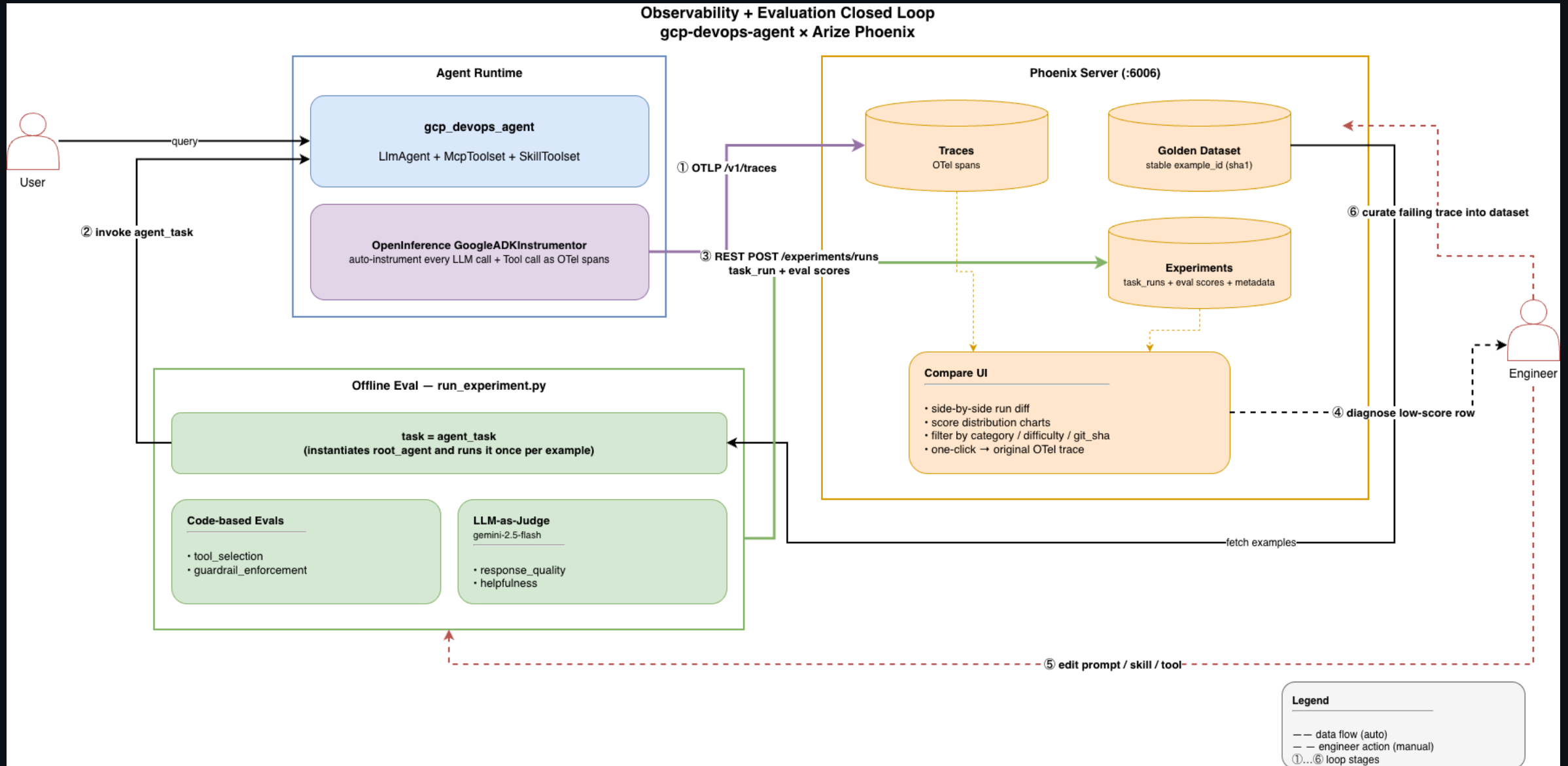
- invocation [InMemoryRunner] 4.83s
 - agent_run [root_agent] 4.83s
 - call_llm @ 927 4.83s
 - execute_tool transfer_to_agent 0.01s
 - agent_run [pre_trip_agent] 3.75s
 - call_llm @ 1052 3.75s
 - execute_tool transfer_to_agent 0s
 - agent_run [pre_trip_agent] 2.58s

The right panel shows the input for the selected span (span_id: 2ab0d9ed36b65916) with a latency of 4.83s. The input is a JSON object:

```
{
  "user_id": "test_user",
  "session_id": "4f287639-b1d5-44d1-9296-90d5e4150a66",
  "new_message": {
    "parts": [
      {
        "video_metadata": null,
        "thought": null,
        "inline_data": null,
        "file_data": null,
        "thought_signature": null,
        "code_execution_result": null,
        "executable_code": null,
        "function_call": null,
        "function_response": null,
        "text": "monitor"
      }
    ]
  },
  "role": "user"
}
```



AI Agent Observability and Evaluation with Arize Phoenix



Thank you

Build agents that you can see, score, and trust.